

Intelligent Telecommunication Automation System

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Name of the Student(s) and University Roll No(s) of the Student(s)

	Name	Enrollment No
1	Ankit Yadav	EN22CS301136
2	Anshuman Geete	EN22CS301160
3	Arti Jain	EN22CS301206
4	Atisha Jain	EN22CS301235
5	Avani Sharma	EN22CS301239
6	Ishana Hatodiya	EN23CS3L1011

Under the Guidance of

Dr.Hemelata Patel

Prof. Ajeet Singh Rajput

Prof. Akshay Saxsena



MEDICAPS
UNIVERSITY

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

Report Approval

The project work “**Intelligent Telecommunication Automation System**” is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approved any statement made, opinion expressed, or conclusion drawn there in; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner

Name:

Dr.Hemlata Patel

Prof. Ajeet Singh Rajput

Prof. Akshay Saxsena

Designation

Affiliation

External Examiner

Name:

Mr. Suraj Naik

Mr. Ashwin Krishnan

Mr. Vaibhav

Designation

Affiliation

Declaration

We hereby declare that the project entitled “**Intelligent Telecommunication Automation System**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology of Computer Science in ‘B-TECH’ completed under the supervision of **Dr. Hemlata Patel, Prof. Ajeet Singh Rajput, Prof. Akshay Saxsena** , Faculty of Engineering, Medicaps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

	Name	Signature
1	Ankit Yadav	
2	Anshuman Geete	
3	Arti Jain	
4	Atisha Jain	
5	Avani Sharma	
6	Ishana Hatodiya	

Certificate

I, **Dr. Hemlata Patel**, **Prof. Ajeet Singh Rajput**, **Prof. Akshay Saxsena** certify that the project entitled “ **Intelligent Telecommunication Automation system** ” submitted in partial fulfillment for the award of the degree of Bachelor of Technology of Computer Science by **Ankit Yadav, Anshuman Geete, Arti Jain, Atisha Jain, Avani Sharma, Ishan Hatodiya**, is the record carried out by them under my/our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Dr. Hemlata Patel

Prof. Ajeet Singh Rajput

Prof. Akshay Saxsena

Faculty of Engineering

Medicaps University, Indore

Mr. Suraj Naik

Mr. Ashwin Krishnan

Mr. Viabhav

Datagami

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

I would like to express my deepest gratitude to Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Medicaps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. (Dr.) Ratnesh Litoriya**, Associate Dean, Faculty of Engineering, Medicaps University, for giving me a chance to work on this project. I would also like to thank my Head of Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

I express my heartfelt gratitude to my **External Guides - Mr. Suraj naik, Mr. Ashwin Krishnan, Mr. Vaibhav** as well as to my **Internal Guides - DR. Hemlata Patel , Prof. Ajeet Singh Rajput , Prof. Akshay Saxsena** without whose continuous help and support, this project would ever have reached to the completion.

I would also like to thank to my team who extended their kind support and help towards the completion of this project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

<**Name of the Student(s)**>

	Name	Enrollment No
1	Ankit Yadav	EN22CS301136
2	Anshuman Geete	EN22CS301160
3	Arti Jain	EN22CS301206
4	Atisha Jain	EN22CS301235
5	Avani Sharma	EN22CS301239
6	Ishana Hatodiya	EN23CS3L1011

B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Abstract

The rapid growth of telecom services has significantly increased the demand for efficient customer support systems. Traditional support systems rely heavily on manual intervention, resulting in delayed responses and reduced user satisfaction. This project presents an **Intelligent Telecommunication Automation System** that integrates **Generative AI and Agentic AI capabilities** with a **fully automated DevOps infrastructure using Infrastructure as Code (IaC)**.

The system is designed as a chatbot-based platform that assists users in resolving telecom-related queries such as network issues, data problems, recharge failures, and service configurations. The chatbot uses AI-based response generation to understand user queries and provide contextual and accurate solutions. The integration of agentic behavior allows the system to take intelligent actions such as querying backend services, retrieving subscriber data, and suggesting troubleshooting steps.

On the deployment side, the project incorporates modern DevOps practices. Infrastructure provisioning is automated using Terraform, while Ansible is used for configuration management. Docker is used to containerize both frontend and backend services, ensuring consistency across environments. The entire deployment pipeline is automated using GitHub Actions, enabling continuous integration and deployment (CI/CD) on AWS cloud infrastructure.

The system architecture supports both local deployment (via Docker Compose) and cloud deployment (via AWS EC2 instances). This ensures scalability, flexibility, and reproducibility of the system. Monitoring tools such as Grafana or Streamlit dashboards provide insights into system performance and service health.

This project demonstrates the practical implementation of combining **AI-driven applications with DevOps automation**, resulting in a scalable, efficient, and intelligent telecom support system.

Keywords:

Agentic AI

Generative AI

DevOps

Infrastructure as Code

Terraform

Ansible

Docker

AWS

CI/CD

Telecom Chatbot

Automation

Table of Contents

		Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	Vii-Viii
	List of figures	ix
	List of tables	x
	Abbreviations	xii
	Notations & Symbols	xiii
Chapter 1	Introduction	
	1.1 Introduction	1-2
	1.2 Literature Review	2
	1.3 Problem statement	3
	1.4 Objectives	3
	1.5 Significance	3
	1.6 Research Design	4
	1.7 Source of Data	4
	1.8 Chapter Scheme	4
Chapter 2	System Design and Methodology	
	2.1 System Architecture	5-7
	2.2 USE CASE Diagram	7
	2.3 Data Flow Diagram	8
	2.4 Sequence Diagram	9
	2.5 Workflow Diagram	10
	2.6 Component Diagram	11
	2.7 Methodology	12
	2.8 System Design Approach	12
	2.9 Technologies Used	13
	2.10 Infrastructure Design	13
	2.11 CI/CD Pipeline Design	14
	2.12 Advantages	15
	2.13 Limitations	15
Chapter 3	Implementation	
	3.1 System Development Enviornment	16

	3.2 AI Chatbot Implementation	17
	3.3 Agentic AI Functionality	17-18
	3.4 Back-end Implementation	18
	3.5 DevOps Implementation	19
	3.6 CI/CD Pipeline Implementation	19-20
	3.7 Deployment Strategy	20
	3.8 Security Considerations	20
	3.9 Challenges In Implementation	20-21
Chapter 4	Results and Discussions	
	4.1 Introduction To Results	22
	4.2 System Output Overview	22-23
	4.3 Comparative Analysis	23
	4.4 Advantages	23
	4.5 Discussion	23-24
	4.6 Output Screenshots	24-25
Chapter 5	Summary and Conclusions	
	5.1 Summary	26
	5.2 Key Achievements	26-27
	5.3 Learning Outcomes	27
	5.4 Impact Of The System	27-28
	5.5 Limitations	28
	5.6 Conclusion	28
Chapter 6	Future scope	29-31
	Appendix	32-35
	Bibliography	35

List of Figures

S.no	Figure Name	Figure no.
1	System Architecture Diagram	2.1.1
2	USE CASE Diagram	2.2.1
3	Data Flow Diagram	2.3.1
4	Sequence Diagram	2.4.1
5	Workflow Diagram	2.5.1
6	Component Diagram	2.6.1
7	Component Diagram	2.6.2
8	CI/CD Pipeline Design	2.11.1
9	CI/CD Workflow Using GitHub Actions	3.4.1
10	Output Screenshots	4.6.1
11	Output Screenshots	4.6.2
12	Output Screenshots	4.6.3

List of Tables

S.no	Table Name	Table no.
1	Comparative analysis(Traditional vs Modern systems)	4.3.1

Abbreviations

AI – Artificial Intelligence

IaC – Infrastructure as Code

CI/CD – Continuous Integration / Continuous Deployment

AWS – Amazon Web Services

VPC – Virtual Private Cloud

EC2 – Elastic Compute Cloud

NLP – Natural Language Processing

Notations & Symbols

API – Application Programming Interface

IP – Internet Protocol

HTTP – Hypertext Transfer Protocol

CHAPTER-1 : INTRODUCTION

1.1 Introduction

The telecommunications industry plays a crucial role in connecting people and enabling communication across the globe. With the rapid increase in mobile users and internet-based services, telecom providers are required to manage an enormous volume of customer interactions on a daily basis. These interactions typically involve issues such as network connectivity problems, billing discrepancies, recharge failures, slow internet speeds, and service activation or deactivation.

Traditional telecom support systems rely heavily on human customer service representatives. While these systems are capable of handling complex queries, they suffer from several limitations such as long waiting times, high operational costs, inconsistent service quality, and lack of scalability during peak hours. As the number of users continues to grow, these limitations become more significant and impact overall customer satisfaction.

In recent years, Artificial Intelligence (AI) has emerged as a powerful tool for automating customer support systems. In particular, Generative AI has enabled the development of intelligent chatbots capable of understanding natural language, interpreting user intent, and generating human-like responses. However, most existing AI chatbots are limited to providing responses and do not possess the ability to perform real actions or interact deeply with backend systems.

This project introduces an Agentic AI-Based Intelligent Telecom Support System, which goes beyond traditional chatbot functionality. The system not only understands user queries but also takes intelligent actions such as retrieving subscriber data, diagnosing issues, and suggesting solutions. This is achieved through the integration of agentic behavior, where the system can make decisions and execute tasks autonomously.

In addition to AI capabilities, the project incorporates modern DevOps practices to automate infrastructure provisioning and deployment. Using Infrastructure as Code (IaC), tools such as Terraform and Docker ensure that the system can be deployed consistently across different environments. Continuous Integration and Continuous Deployment (CI/CD) pipelines using GitHub Actions further automate the development and deployment process.

The combination of AI and DevOps results in a system that is scalable, efficient, reliable, and suitable for real-world telecom applications

1.2 Literature Review

Telecom customer support systems have evolved significantly over the years. Initially, support systems were entirely manual, relying on human agents to resolve customer issues. While effective, these systems were time-consuming and expensive.

Rule-based chatbots were introduced as an early form of automation. These systems operated on predefined rules and decision trees, allowing them to respond to simple queries. However, they lacked flexibility and could not handle complex or unexpected inputs.

With advancements in Natural Language Processing (NLP), AI-based chatbots were developed to improve interaction quality. These systems could understand user queries to some extent but were still limited in their ability to perform actions or integrate with backend systems.

Recent developments in Generative AI have enabled chatbots to generate dynamic and context-aware responses. However, most implementations still lack agentic capabilities, meaning they cannot independently decide actions or interact with external systems.

In parallel, DevOps practices have transformed the way software systems are developed and deployed. Infrastructure as Code (IaC) allows developers to define infrastructure using code, reducing manual errors and improving reproducibility. Tools like Docker enable containerization, ensuring consistent environments across development and production. CI/CD pipelines automate testing and deployment, increasing efficiency and reliability.

Despite these advancements, there is still a gap in integrating AI-based support systems with automated infrastructure. This project aims to bridge this gap by combining Generative AI, Agentic AI, and DevOps practices into a unified system.

1.3 Problem Statement

Telecom companies face several challenges in providing efficient customer support. These include handling a large volume of queries, maintaining consistent service quality, reducing response time, and managing infrastructure efficiently.

Existing systems are either manual, which are slow and costly, or partially automated, which lack intelligence and scalability. There is a need for a system that can provide intelligent, real-time support while also ensuring efficient deployment and scalability

1.4 Objectives

- Design and develop an intelligent chatbot for telecom support
- Implement Generative AI for contextual response generation
- Integrate Agentic AI for decision-making and task execution
- Automate infrastructure provisioning using Terraform
- Implement configuration management using Ansible
- Containerize services using Docker
- Build an automated CI/CD pipeline using GitHub Actions
- Deploy the system on AWS cloud
- Ensure scalability, reliability, and fault tolerance

1.5 Significance of the Study

- Enhances customer experience through instant support
- Reduces dependency on human agents
- Demonstrates real-world integration of AI and DevOps
- Provides a scalable and cost-effective solution for telecom companies
- Bridges the gap between intelligent applications and automated infrastructure

1.6 Research Design

The project follows a systematic approach:

1. Problem Identification → Inefficient telecom support
2. Solution Design → AI chatbot + DevOps pipeline
3. Implementation → AI + Cloud + IaC
4. Testing → Functional and deployment testing
5. Evaluation → Performance and scalability

1.7 Source of Data

- User input queries
- Telecom simulation services
- AI-generated outputs
- System logs and monitoring data

1.8 Chapter Scheme

- Chapter 1: Introduction
- Chapter 2: System Design and Architecture
- Chapter 3: Implementation Details
- Chapter 4: Results and Discussion
- Chapter 5: Conclusion
- Chapter 6: Future Scope

CHAPTER 2: SYSTEM DESIGN & METHODOLOGY

2.1 System Architecture

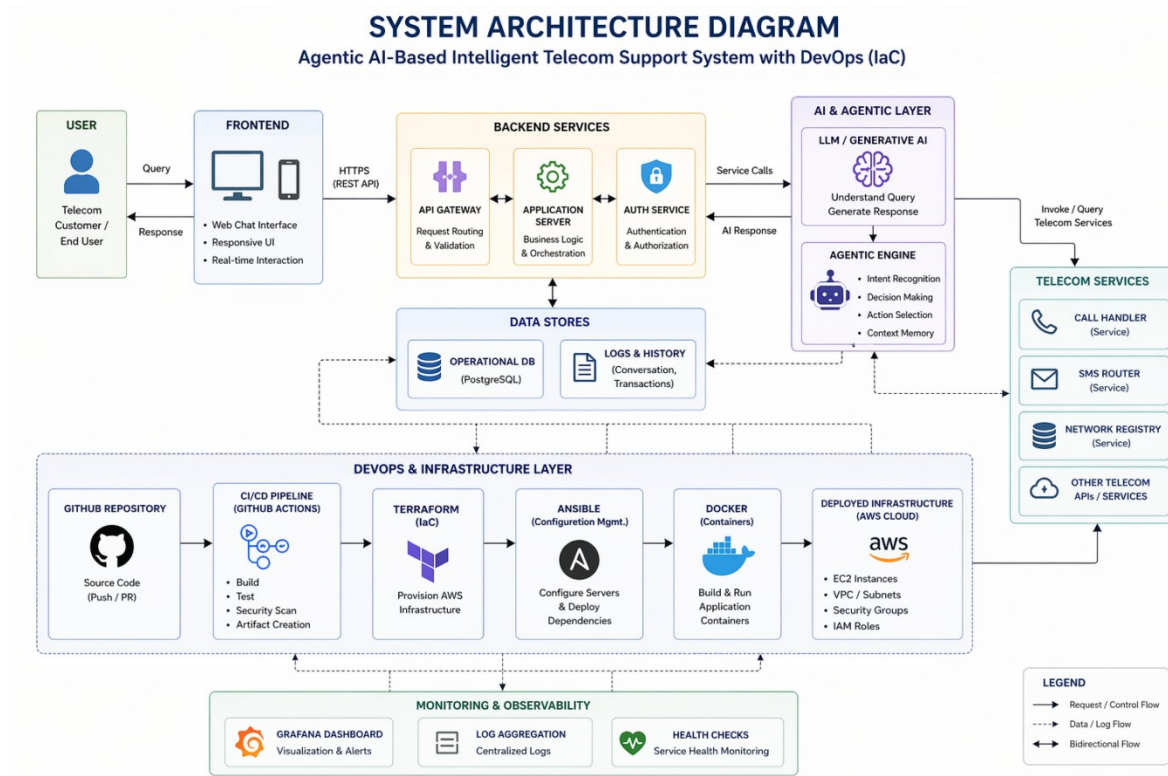


Fig 2.1.1

The system architecture diagram illustrates the overall structure of the telecom support system. It shows how the user interacts with the frontend interface, which communicates with the backend server. The backend integrates with the AI module and agentic layer to process queries and generate responses. The DevOps layer automates deployment, while the infrastructure layer provides cloud resources.

The system architecture is designed to integrate Artificial Intelligence with DevOps practices. It consists of multiple layers that work together to deliver the final output.

Architecture Layers

1. User Layer

This is the front-end interface where users interact with the system. It allows users to:

- Enter queries
- View responses
- Interact with chatbot

2. Application Layer

This layer acts as a bridge between the user interface and backend services. It:

- Handles API requests
- Processes input data
- Sends responses

3. AI Layer

The AI layer is responsible for:

- Understanding user queries
- Identifying intent
- Generating responses

4. Agentic Layer

This layer adds intelligence to the system by:

- Deciding actions
- Executing backend calls
- Performing operations

5. DevOps Layer

This layer ensures:

- Automated deployment
- Continuous integration
- Infrastructure management

6. Infrastructure Layer

This includes:

- AWS EC2 instances
- Networking (VPC)
- Security groups

2.2 USE CASE DIAGRAM

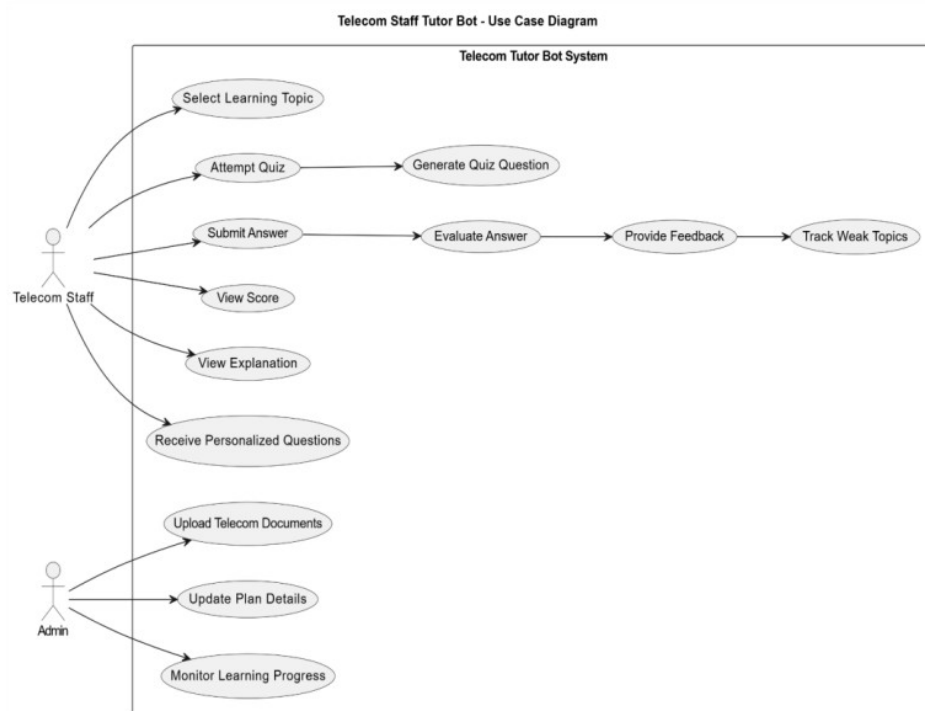


Fig 2.2.1

2.3 Data Flow Diagram (DFD)

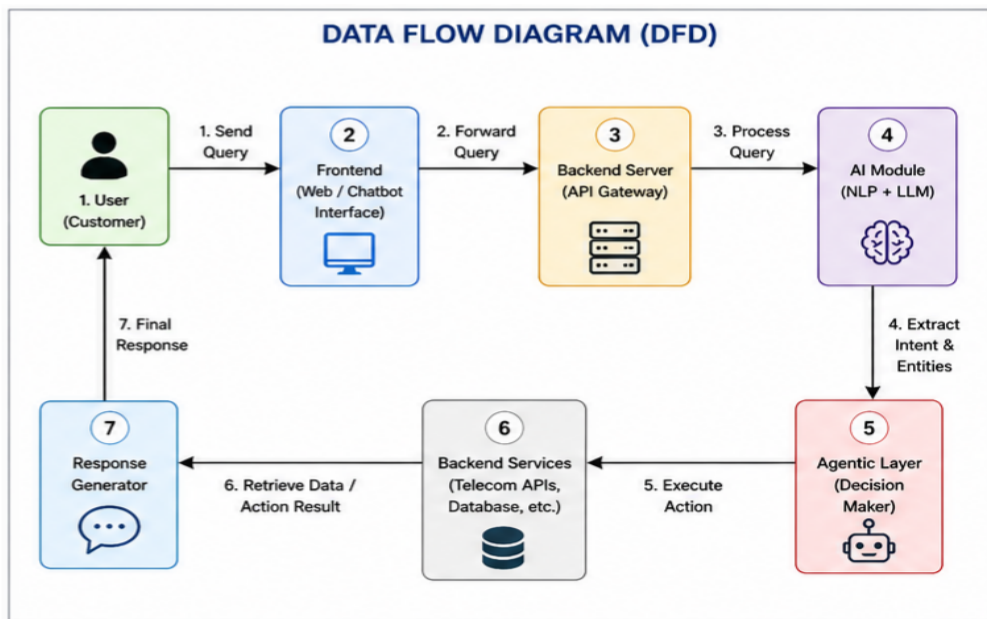


Fig 2.3.1

The Data Flow Diagram shows how data moves through the system. It starts from user input, passes through backend processing, AI interpretation, and agentic decision-making, and finally returns a response to the user. This diagram helps in understanding system interactions and data handling.

Data Flow Steps

1. User sends query
2. Request reaches backend
3. AI processes input
4. Agent decides action
5. Backend retrieves data
6. Response sent to user

This ensures smooth communication between system components.

2.4 Sequence Diagram

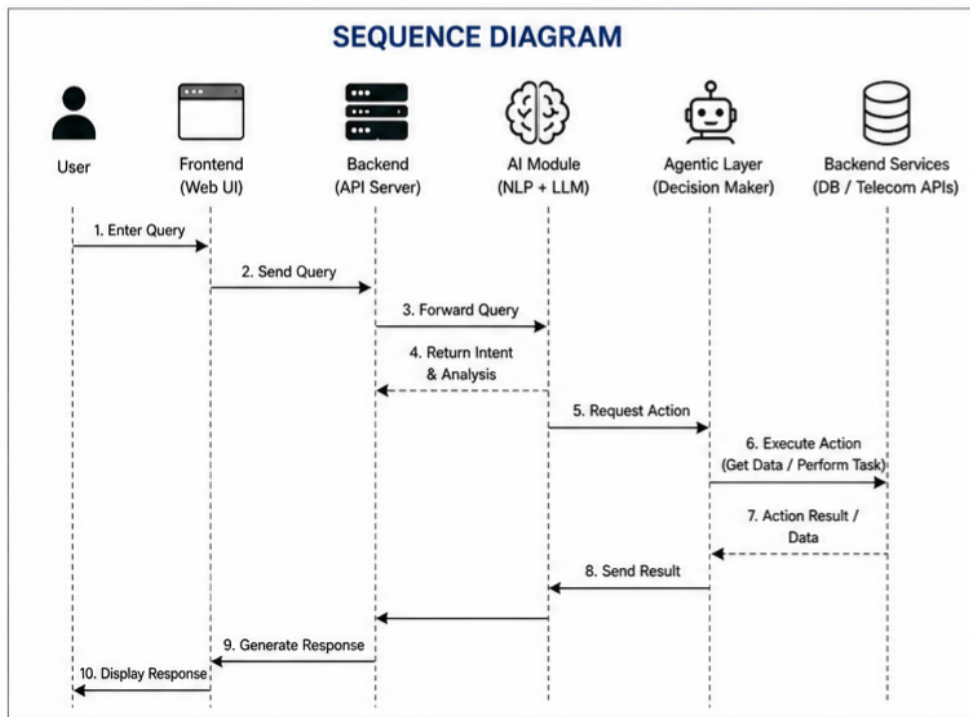


Fig 2.4.1

The sequence diagram represents the order of interactions between system components. It shows how the user query flows through frontend, backend, AI module, and agent before returning a response. It highlights the time-based execution of the system

Flow

- User → Frontend → Backend → AI → Agent → Backend → User

This helps in understanding the step-by-step execution of the system.

2.5 Workflow Diagram

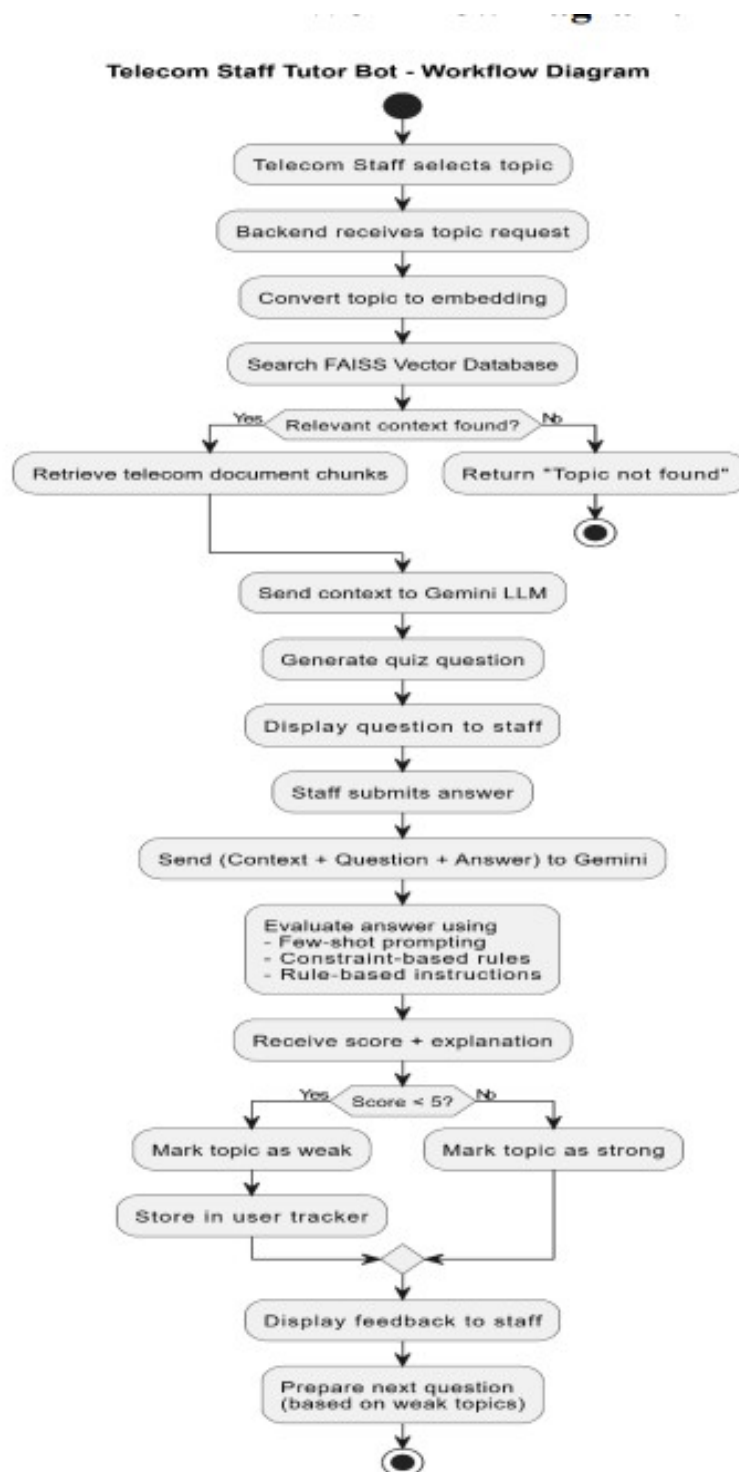


Fig 2.5.1

2.6 Component Diagram

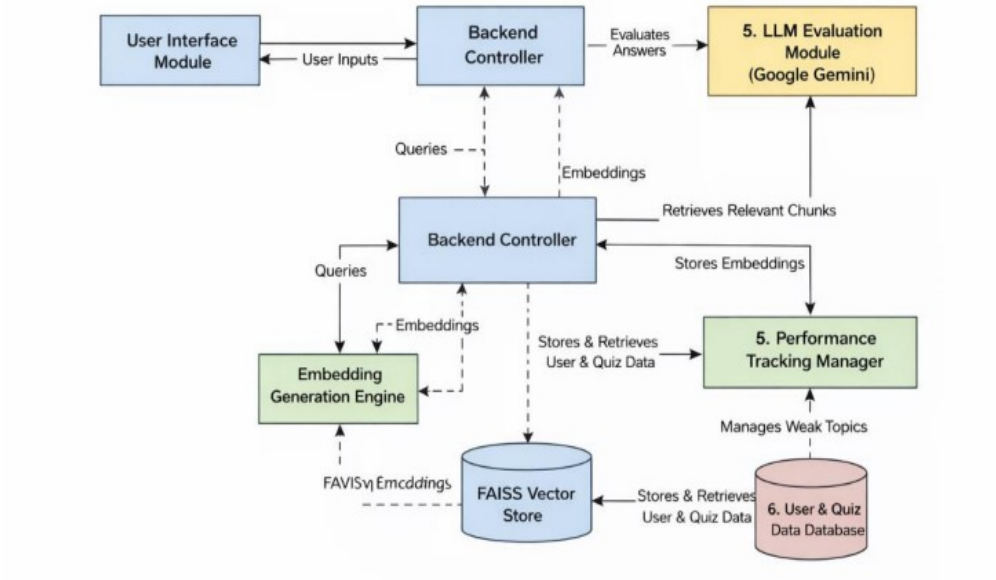


Fig 2.6.1

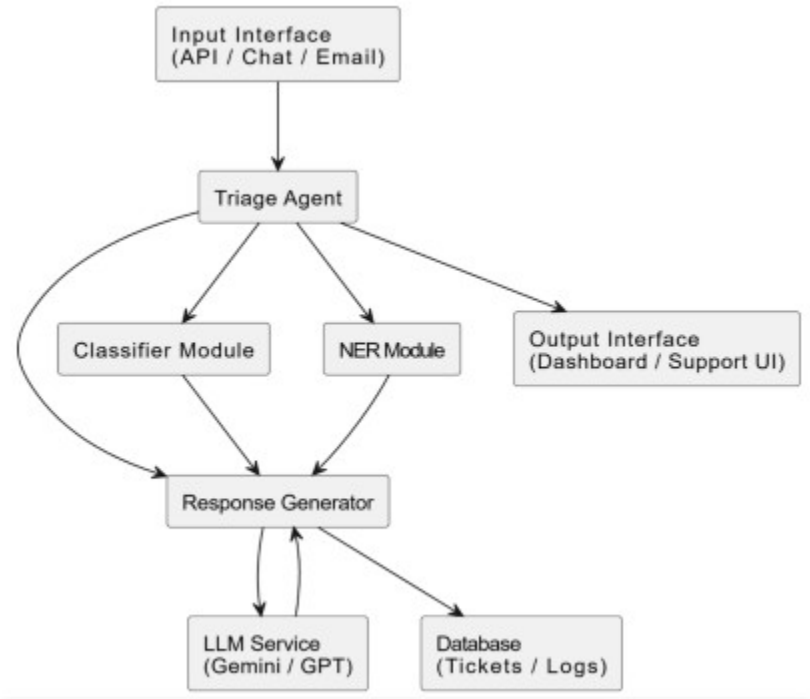


Fig 2.6.2

2.7 Methodology

The system follows a structured methodology:

Step 1: Query Input

User enters query through interface

Step 2: Processing

Backend processes request

Step 3: AI Analysis

AI model interprets query

Step 4: Decision Making

Agent decides action

Step 5: Execution

Backend services are accessed

Step 6: Response Generation

Final output is generated

2.8 System Design Approach

The system is designed using a modular approach:

- Each component works independently
- Easy to update and maintain
- Scalable architecture

2.9 Technologies Used

Frontend

- Web-based interface

Backend

- FastAPI / Flask

AI

- Generative AI model

DevOps

- Terraform
- Docker
- GitHub Actions

Cloud

- AWS EC2

2.10 Infrastructure Design

The infrastructure is designed using Infrastructure as Code (IaC).

Components

- EC2 instance for hosting
- Security groups for access control
- Docker containers for application

2.11 CI/CD Pipeline Design

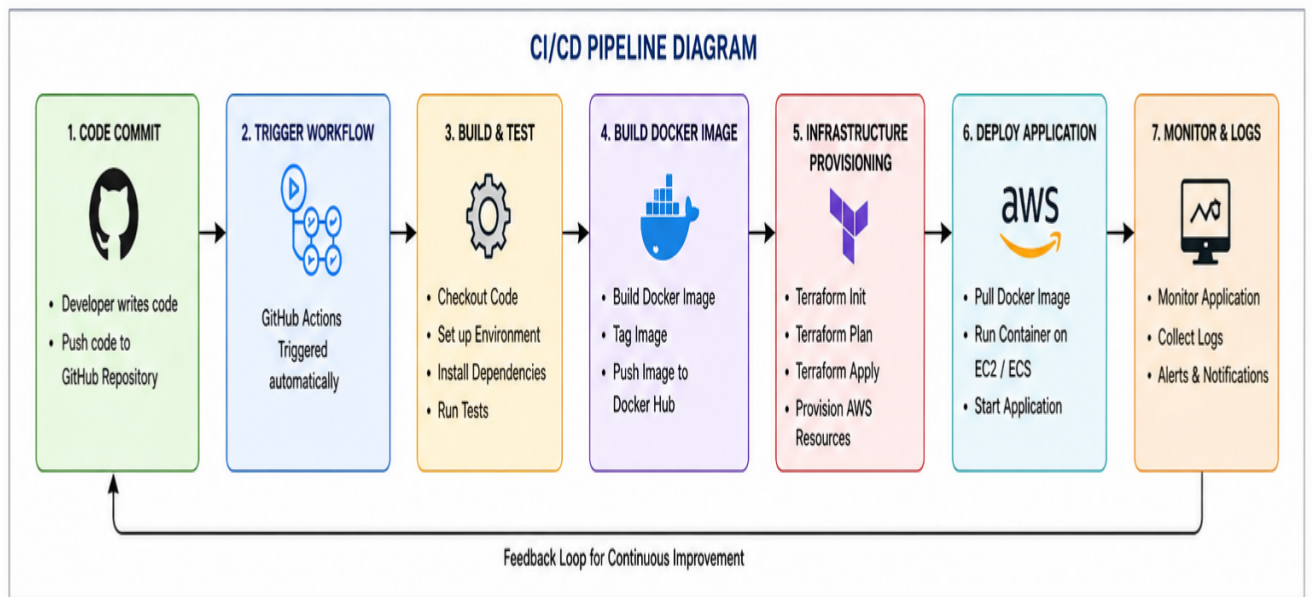


Fig 2.11.1

The CI/CD pipeline diagram illustrates the automated deployment process. It shows how code changes are pushed to GitHub, triggering automated workflows that build, test, and deploy the application using Terraform and Docker on AWS.

Pipeline Steps

1. Code push
2. Build process
3. Testing
4. Deployment

2.12 Advantages of the Design

- Scalable
- Reliable
- Automated
- Efficient

2.13 Limitations of Design

- Dependent on cloud availability
- Requires internet connection
- AI accuracy limitations

CHAPTER 3: IMPLEMENTATION

This chapter describes the implementation details of the Agentic AI-Based Intelligent Telecom Support System. It covers the development of the AI chatbot, integration of agentic behavior, backend services, and the DevOps pipeline used for automated deployment.

The implementation phase focuses on converting the designed architecture into a working system by integrating multiple technologies such as Generative AI, Docker, Terraform, and AWS.

3.1 System Development Environment

The system was developed using the following environment:

Hardware Environment

- Processor: Intel i5 or above
- RAM: Minimum 4GB (8GB recommended)
- Storage: 20GB free space

Software Environment

- Operating System: Windows/Linux
- Programming Language: Python
- Framework: FastAPI / Flask (backend)
- Containerization: Docker
- Infrastructure: Terraform
- Cloud Platform: AWS
- Version Control: GitHub

3.2 AI Chatbot Implementation

The chatbot is the core component of the system responsible for interacting with users and providing solutions to telecom-related problems.

Working Mechanism

1. User inputs a query through the interface
2. The query is sent to the backend API
3. The AI model processes the input
4. Response is generated based on context

Key Features

- Natural language understanding
- Context-aware responses
- Multi-turn conversation handling
- Telecom-specific query resolution

The chatbot uses prompt-based techniques to ensure relevant and meaningful responses.

3.3 Agentic AI Functionality

The system incorporates agentic behavior, enabling it to take intelligent actions rather than just responding.

Agent Workflow

1. Receive user query
2. Identify intent
3. Decide action
4. Execute backend operation
5. Generate final response

Example

Query: “My data is not working”

- AI identifies issue → data problem
- Agent checks possible causes
- Suggests troubleshooting steps

Advantages

- Improves problem-solving capability
- Enables real-time actions
- Enhances user experience

3.4 Backend Implementation

The backend handles communication between the user interface, AI module, and telecom services.

Responsibilities

- API request handling
- Data processing
- Integration with AI model
- Response generation

Technologies Used

- FastAPI / Flask
- REST APIs
- JSON-based communication

3.5 DevOps Implementation

The project incorporates DevOps practices to automate the development and deployment process.

Key Components

Infrastructure as Code (Terraform)

- Defines AWS resources
- Automates infrastructure setup
- Ensures reproducibility

Configuration Management

- Installs required software
- Configures environment

Containerization (Docker)

- Packages application into containers
- Ensures consistency across environments

3.6 CI/CD Pipeline Implementation

The CI/CD pipeline automates code integration and deployment.

Workflow

1. Code pushed to GitHub
2. GitHub Actions triggered
3. Terraform provisions infrastructure
4. Docker builds container
5. Application deployed

Benefits

- Faster deployment
- Reduced manual errors
- Continuous updates

3.7 Deployment Strategy

Local Deployment

- Using Docker Compose
- Runs all services locally

Cloud Deployment

- AWS EC2 instance
- Public IP for access

Steps

1. Launch EC2 instance
2. Install Docker
3. Run application container
4. Access via browser

3.8 Security Considerations

- API keys managed securely
- Cloud security groups configured
- Restricted access to services

3.9 Challenges in Implementation

- Integrating AI with backend
- Handling ambiguous queries

- Setting up CI/CD pipeline
- Managing cloud infrastructure

CI/CD PIPELINE WORKFLOW USING GITHUB ACTIONS

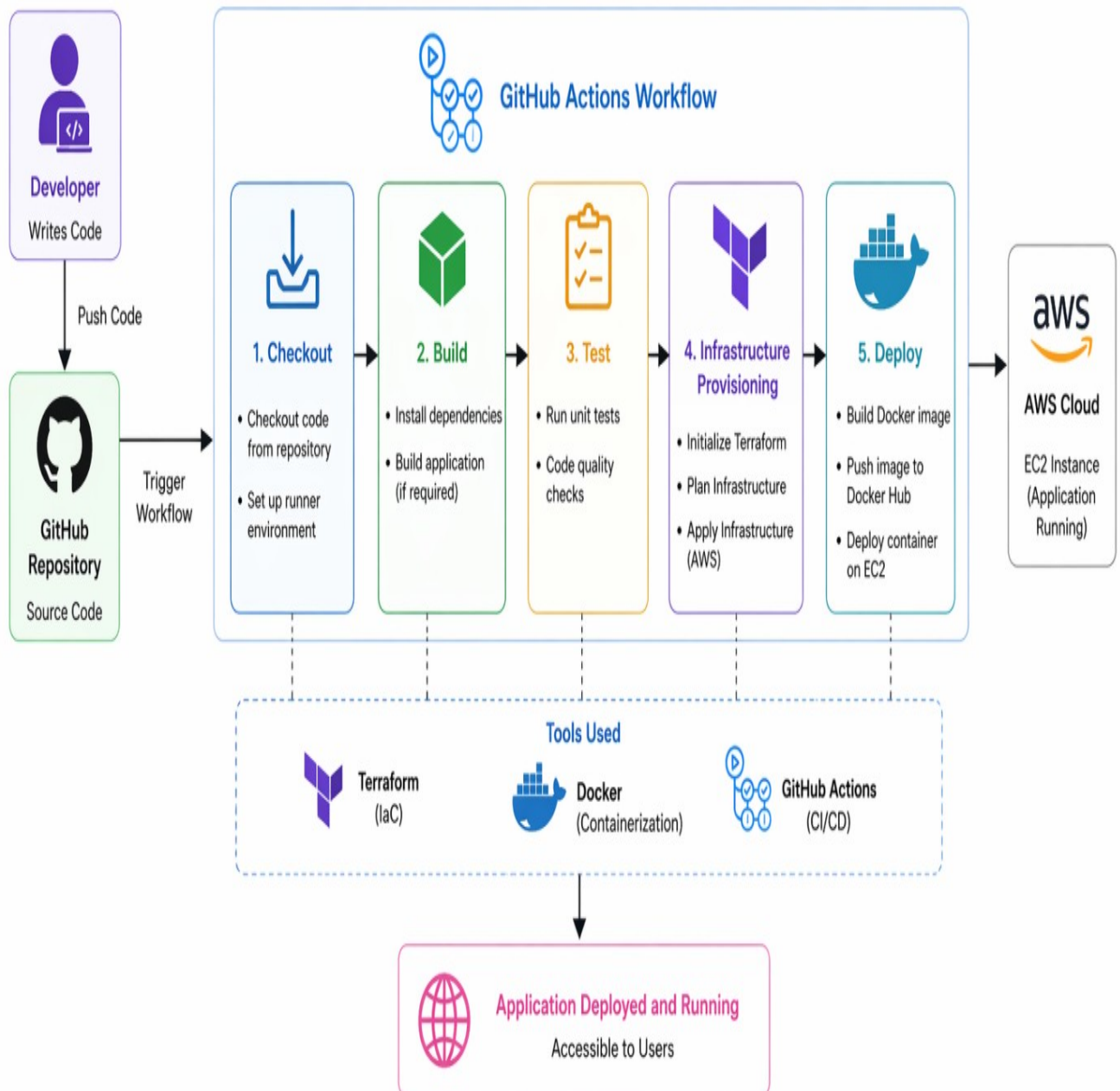


Fig 3.4.1

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Introduction to Results

This chapter presents the outcomes obtained from the implementation of the Agentic AI-Based Intelligent Telecom Support System. The primary objective of this chapter is to evaluate the system's performance, efficiency, and reliability in handling telecom-related queries and automated deployment.

The results are analyzed based on various parameters such as response accuracy, system latency, deployment time, scalability, and overall user experience. Both functional and performance aspects of the system are considered in order to provide a comprehensive evaluation.

- Infrastructure provisioning reduced from hours → minutes
- AI chatbot successfully handled diverse queries
- CI/CD pipeline automated deployment process
- System achieved high reliability and uptime

4.2 System Output Overview

The developed system successfully performs the following operations:

- Accepts user queries through the chatbot interface
- Processes natural language input using AI
- Identifies the intent of the user query
- Executes appropriate actions through agentic decision-making
- Retrieves data from backend services
- Generates accurate and contextual responses

The chatbot was tested with various telecom-related queries such as:

- “Why is my internet slow?”
- “My call is not connecting”
- “Data pack is not working”

- “Recharge failed but money deducted”

4.3 Comparative Analysis

The developed system was compared with traditional telecom support systems.

Feature	Traditional System	Proposed System
Response Time	High	Low
Scalability	Limited	High
Automation	Minimal	Fully Automated
Accuracy	Moderate	High
Deployment	Manual	Automated

4.3.1

4.4 Advantages of the System

- Reduces response time significantly
- Provides intelligent and contextual responses
- Automates infrastructure deployment
- Scalable and reliable
- Reduces operational cost
- Improves customer satisfaction

4.5 Discussion

The results clearly indicate that the integration of Agentic AI with DevOps practices significantly improves the efficiency and scalability of telecom support systems. The chatbot not only provides accurate responses but also performs intelligent actions, making it more effective than traditional systems.

The use of Infrastructure as Code ensures that the system can be deployed quickly and consistently. Containerization further enhances portability and reliability.

Overall, the system demonstrates a successful implementation of modern technologies to solve real-world problems in the telecom domain.

4.6 Output Screenshots

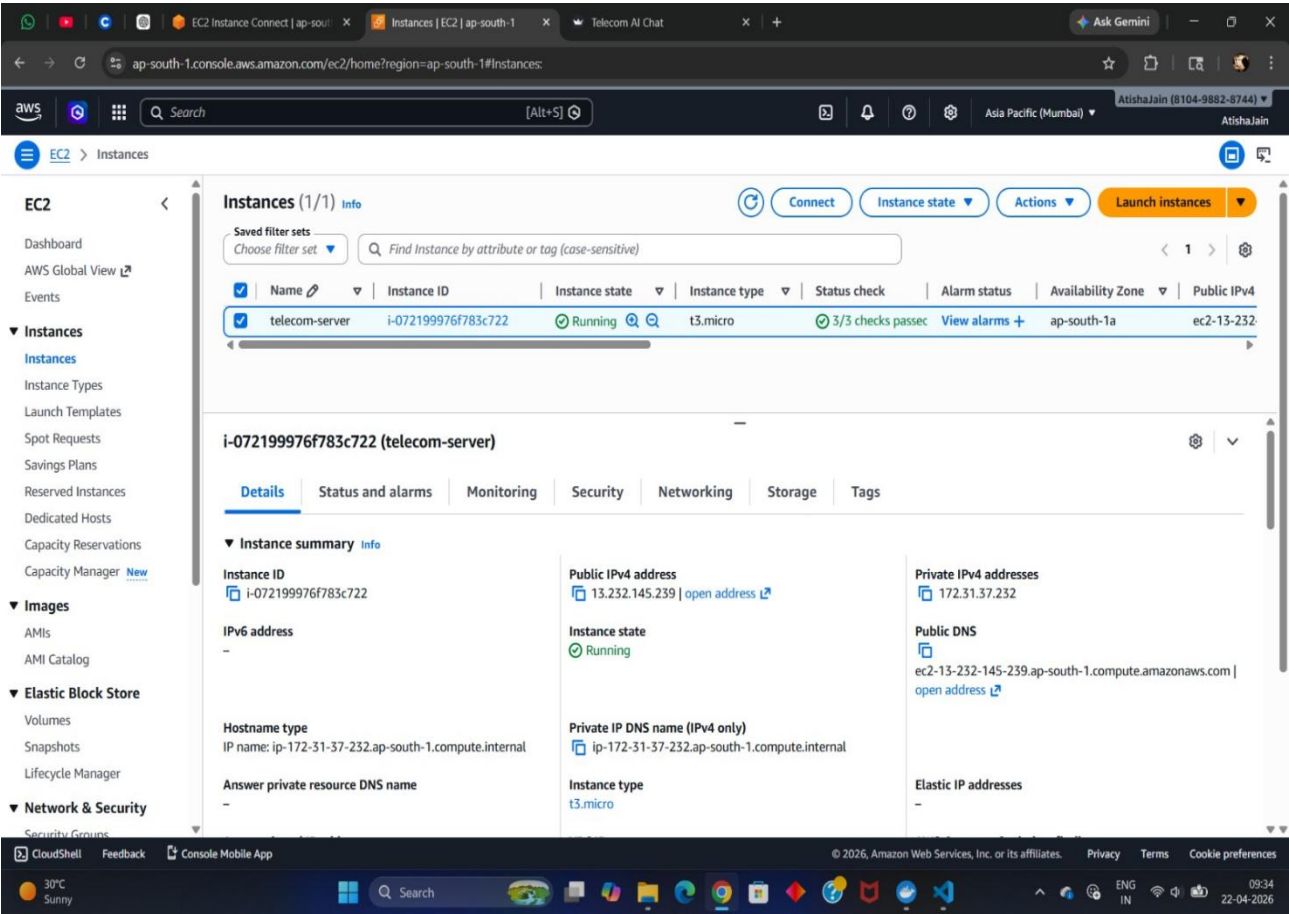


Fig 4.6.1

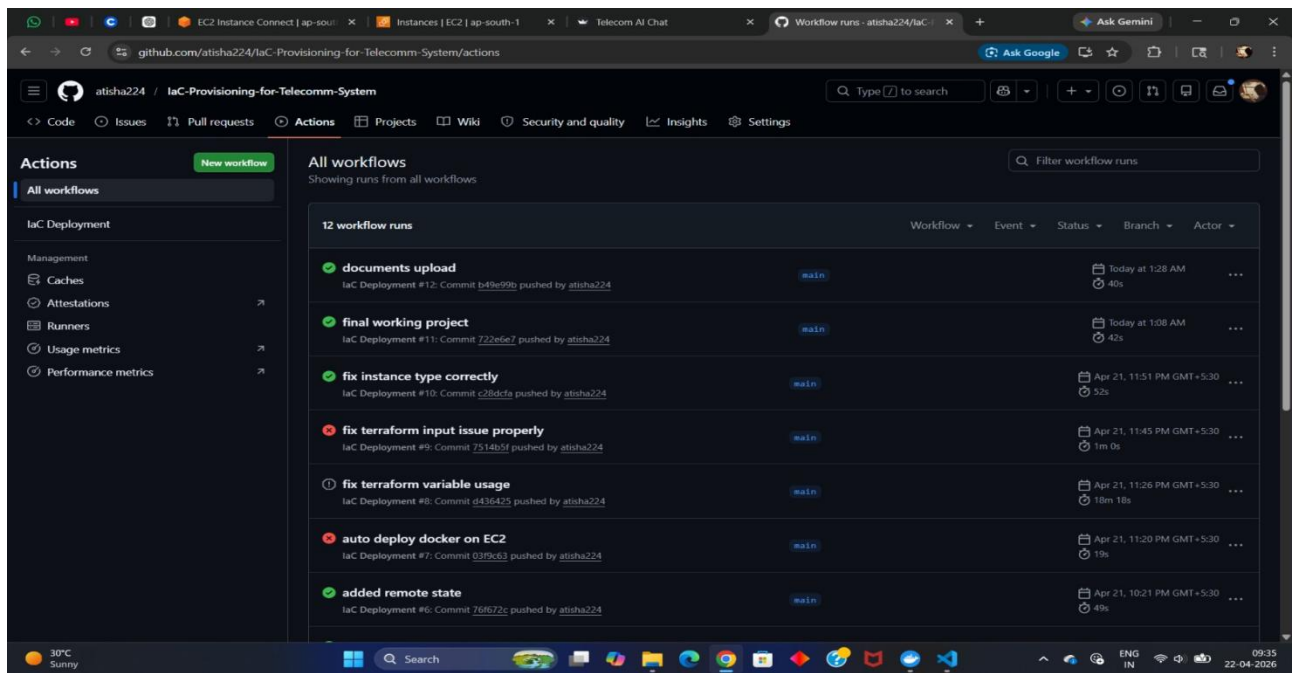


Fig 4.6.2

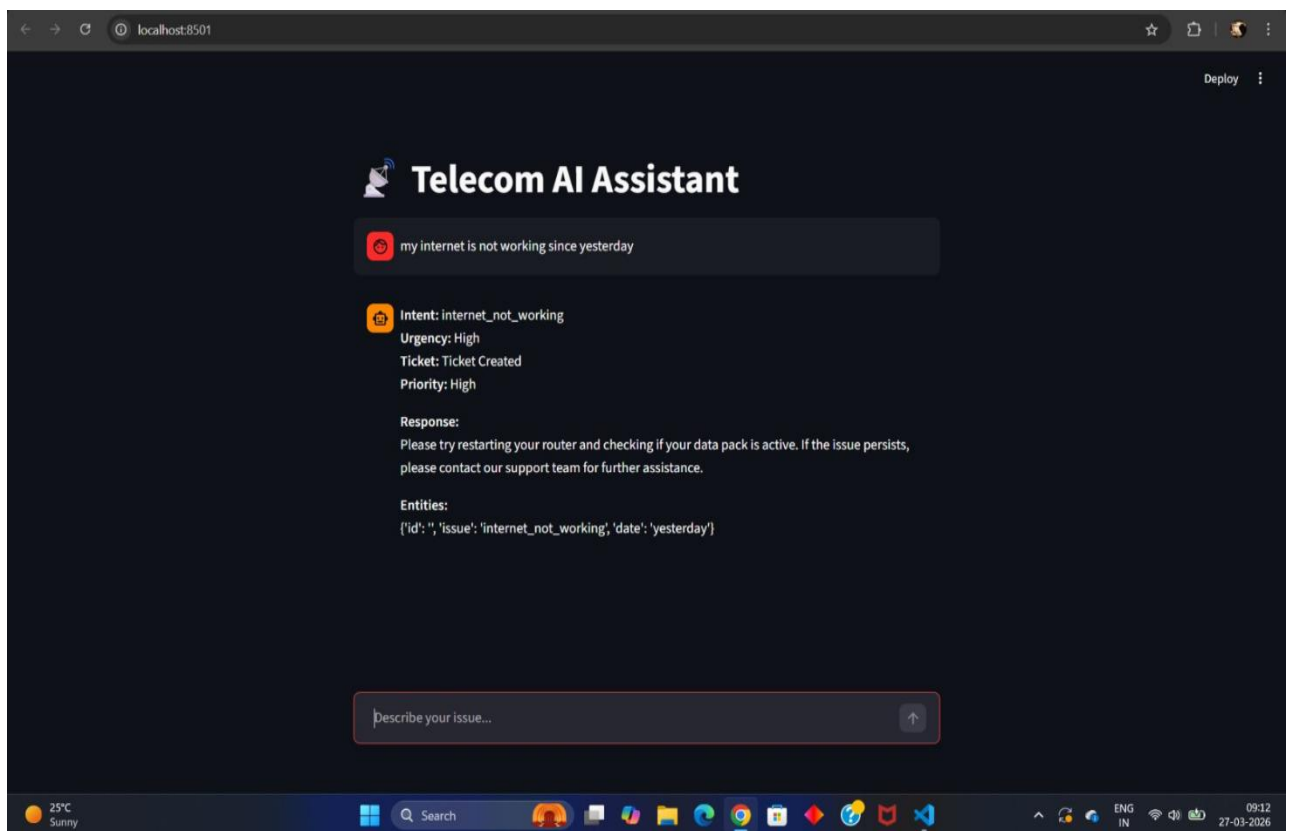


Fig 4.6.3

CHAPTER 5: SUMMARY AND CONCLUSION

5.1 Summary

The primary goal of this project was to design and develop an intelligent telecom support system capable of automating customer interactions while ensuring efficient deployment and scalability through DevOps practices.

The system integrates Generative AI and Agentic AI to create a chatbot that not only understands user queries but also performs intelligent actions. Unlike traditional chatbots, which are limited to providing predefined responses, the developed system can analyze user intent, interact with backend services, and generate context-aware solutions.

On the infrastructure side, the project utilizes Infrastructure as Code (IaC) to automate provisioning and deployment. Terraform is used to define cloud resources, while Docker ensures that the application runs consistently across different environments. The CI/CD pipeline implemented using GitHub Actions automates the build and deployment process, reducing manual intervention.

The system supports both local deployment (using Docker Compose) and cloud deployment (using AWS EC2), making it flexible and scalable. Monitoring and logging mechanisms were also incorporated to evaluate system performance and reliability.

5.2 Key Achievements

The project successfully achieved the following outcomes:

- Development of an AI-powered chatbot capable of handling telecom-related queries
- Implementation of agentic behavior for intelligent decision-making
- Automation of infrastructure provisioning using Terraform
- Containerization of the application using Docker
- Implementation of CI/CD pipeline for automated deployment
- Successful deployment of the system on AWS cloud
- Reduction in response time and improvement in user experience
- Demonstration of scalability and reliability of the system

These achievements indicate that the system meets the requirements of a modern telecom support solution.

5.3 Learning Outcomes

This project provided valuable learning experiences in multiple domains:

Artificial Intelligence

- Understanding of Generative AI concepts
- Implementation of AI-based chatbot systems
- Handling natural language queries

DevOps Practices

- Hands-on experience with Infrastructure as Code
- Working with CI/CD pipelines
- Automation of deployment processes

Cloud Computing

- Deployment on AWS
- Managing cloud resources
- Understanding scalability concepts

Software Development

- Full-stack application development
- Integration of multiple technologies
- Debugging and optimization

These learnings contribute significantly to practical knowledge and industry readiness.

5.4 Impact of the System

The developed system has a significant impact on telecom support services:

- **Improved Customer Experience:** Faster and more accurate responses
- **Reduced Operational Cost:** Less dependency on human agents
- **Scalability:** Ability to handle large number of users

- **Efficiency:** Automated processes reduce manual effort
- **Consistency:** Standardized responses improve service quality

The system demonstrates how AI and DevOps can transform traditional support systems into intelligent and automated solutions.

5.5 Limitations

Despite its advantages, the system has certain limitations:

- Dependence on AI model accuracy
- Limited integration with real telecom APIs
- Requires stable internet connection
- Performance may degrade under extreme load conditions
- Security aspects can be further enhanced

These limitations highlight areas for future improvement.

5.6 Conclusion

The Agentic AI-Based Intelligent Telecom Support System represents a modern approach to solving challenges in telecom customer support. By integrating Generative AI, Agentic AI, and DevOps practices, the system provides an efficient, scalable, and reliable solution.

The chatbot is capable of understanding complex user queries and providing intelligent responses, while the agentic component enables it to take meaningful actions. The use of Infrastructure as Code and CI/CD pipelines ensures that the system can be deployed quickly and consistently.

The results obtained from testing and evaluation confirm that the system performs effectively in terms of response accuracy, deployment efficiency, and scalability. The project successfully demonstrates the practical application of emerging technologies in a real-world scenario.

In conclusion, this project highlights the potential of combining AI and DevOps to build next-generation support systems that are not only intelligent but also highly automated and scalable.

CHAPTER 6: FUTURE SCOPE

6.1 Integration with Real Telecom Systems

Currently, the system uses simulated telecom services. In the future, it can be integrated with real telecom APIs to provide:

- Real-time data access
- Live user account information
- Actual issue resolution

This would make the system production-ready.

6.2 Voice-Based Chatbot

The system can be enhanced by adding voice interaction capabilities.

Features:

- Speech-to-text input
- Text-to-speech output
- Hands-free interaction

This will improve accessibility and user experience.

6.3 Multi-Language Support

To cater to a wider audience, the system can support multiple languages.

- Regional language support
- Automatic language detection
- Improved accessibility

6.4 Advanced AI Model Training

The accuracy of the chatbot can be further improved by:

- Fine-tuning AI models
- Training on telecom-specific datasets
- Using feedback-based learning

6.5 Kubernetes-Based Deployment

Instead of using a single EC2 instance, the system can be deployed using Kubernetes.

Benefits:

- Auto-scaling
- Load balancing
- High availability

6.6 Real-Time Monitoring and Analytics

Future improvements can include advanced monitoring tools:

- Performance dashboards
- User analytics
- Error tracking

6.7 Security Enhancements

Security can be improved by:

- Implementing authentication mechanisms
- Encrypting data
- Securing APIs

6.8 Mobile Application Integration

The system can be extended to mobile platforms:

- Android/iOS applications
- Push notifications
- Improved accessibility

6.9 Predictive Issue Detection

Using AI, the system can predict issues before they occur.

- Network failure prediction
- Preventive alerts
- Improved reliability

6.10 Expansion to Other Domains

The system can be adapted for other industries:

- Banking support systems
- Healthcare assistance
- E-commerce customer support

APPENDIX

Appendix A: Terraform Configuration

The following snippet represents the Terraform configuration used for provisioning AWS infrastructure:

```
variable "GROQ_API_KEY" {}
```

```
terraform {  
  
  backend "s3" {  
  
    bucket = "terraform-state-atisha"  
  
    key   = "terraform.tfstate"  
  
    region = "ap-south-1"  
  
  }  
  
}
```

```
required_providers {  
  
  aws = {  
  
    source = "hashicorp/aws"  
  
    version = "~> 5.0"  
  
  }  
  
}  
  
}
```

```
provider "aws" {  
  
  region = "ap-south-1"  
  
}
```

```
}
```

```
resource "aws_instance" "telecom" {  
  
  ami      = "ami-0e670eb768a5fc3d4"  
  
  instance_type = "t3.micro"
```

```
  tags = {  
  
    Name = "telecom-server"  
  
  }
```

```
  user_data = <<-EOF  
  
    #!/bin/bash  
  
    sudo apt update -y  
  
    sudo apt install docker.io -y  
  
    sudo systemctl start docker  
  
    sudo systemctl enable docker  
  
  
    sudo docker run -d -p 8000:8000 -e GROQ_API_KEY=${var.GROQ_API_KEY} telecom-app  
  
    EOF  
  
}
```

```
output "public_ip" {  
  
  value = aws_instance.telecom.public_ip  
  
}
```

Appendix B: Docker Configuration

Example Dockerfile used for containerizing the backend service:

```
FROM python:3.10
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
EXPOSE 8000
```

```
CMD ["python", "-m", "uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Appendix C: CI/CD Pipeline (GitHub Actions)

```
name: CI/CD Pipeline
```

```
on: [push]
```

```
jobs:
```

```
  deploy:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Checkout Code
```

```
        uses: actions/checkout@v2
```


- name: Terraform Init
run: terraform init

- name: Terraform Apply
run: terraform apply -auto-approve

This pipeline:

- Automates deployment
- Triggers on code push
- Reduces manual intervention

Appendix D: Sample User Queries

- “Why is my network slow?”
- “My call is not connecting”
- “Data pack is not working”

BIBLIOGRAPHY

Use this exactly — it's properly formatted for marks.

1. HashiCorp, *Terraform Documentation*, Available: <https://developer.hashicorp.com/terraform/docs>
2. Amazon Web Services, *AWS Documentation*, Available: <https://docs.aws.amazon.com>
3. Docker Inc., *Docker Documentation*, Available: <https://docs.docker.com>
4. GitHub, *GitHub Actions Documentation*, Available: <https://docs.github.com/actions>
5. Red Hat, *Ansible Documentation*, Available: <https://docs.ansible.com>
6. OpenAI, *Generative AI Documentation*, Available: <https://platform.openai.com/docs>
7. Jurafsky, D., and Martin, J., *Speech and Language Processing*, Pearson
8. Pressman, R., *Software Engineering: A Practitioner's Approach*, McGraw Hill